Game Programming

Fachhochschule Oberösterreich

Agenda

1. Advanced Game Programming Concepts
2. Delegates
3. Lambdas
4. Events

Game Programming

Lecture Goals

- Understand why large game projects need modular architecture
- Use delegates, events, and lambdas in Unity-oriented C#
- Recognize when event-driven communication improves maintainability
- Apply commands, event queues, components, and data-driven design
- Avoid common architecture pitfalls in student game projects

Game Programming

Why Advanced Game Programming?

- Modern games combine input, physics, UI, audio, animation, AI, save data, and rendering
- These systems must react to the player while staying maintainable
- A small prototype can tolerate messy scripts; a semester project usually cannot
- Advanced C# concepts help us separate responsibilities without losing performance
- Good architecture makes later features cheaper to add

Game Programming

Typical Beginner Problem

- One script controls movement, health, UI, sound, particles, scoring, and game-over logic
- The script grows whenever a new feature is added
- A bug in one feature can affect unrelated behavior
- The object becomes difficult to reuse in a different scene or game mode
- This is the point where **architecture** starts to matter

Game Programming

Problematic Player Script

- The player script directly controls multiple unrelated systems
- This works in a tiny prototype but creates **tight coupling**
- Each new reaction forces a change in the player class

- **What is wrong here?**
  - The player knows about UI representation
  - The player knows about audio playback
  - The player knows about game-over flow
  - Reusing this player requires dragging many scene references
  - The class has more than one reason to change

```
using UnityEngine;
using UnityEngine.UI;

public class Player : MonoBehaviour
{
    public int health = 100;
    public Text healthText;
    public AudioSource damageSound;
    public GameObject gameOverScreen;

    public void TakeDamage(int amount)
    {
        health -= amount;
        healthText.text = "Health: " + health;
        damageSound.Play();

        if (health <= 0)
        {
            gameOverScreen.SetActive(true);
            Time.timeScale = 0f;
        }
    }
}
```

Game Programming

Coupling

- Coupling describes **how strongly one class depends on another class**
- Tight coupling means implementation details are directly known
- Loose coupling means systems communicate through narrow interfaces or events
- Loose coupling is not about removing all references; it is about controlling dependencies
- Unity projects often become tightly coupled through Inspector references

Game Programming

Tight Coupling Example

- The enemy explicitly depends on ScoreManager
- This seems convenient, but the enemy now knows too much about game rules
- Adding achievements, audio, or analytics requires more direct references

- **Problems:**
  - Enemy must know that a ScoreManager exists
  - The scene must provide a reference
  - Reusing the enemy in another game mode is harder

```
using UnityEngine;

public class Enemy : MonoBehaviour
{
    [SerializeField]
    private ScoreManager scoreManager;

    public void Die()
    {
        scoreManager.AddScore(100);
        Destroy(gameObject);
    }
}
```

Game Programming

Coupling

- Instead of asking: Which object should I call directly?
  - **Ask: Which meaningful game event happened?**
  - Then ask: Which systems should **react** to that event?

- This perspective is the foundation of **event-driven gameplay architecture**
- The object that causes the event should not know every possible reaction
  - In our example: *The enemy does not need to know who cares about its death. The enemy only announces that it died.*

Game Programming

Event-Based Game Logic

- The enemy announces that it died
  - The score system may react
  - The audio system may react
  - The achievement system may react
  - The enemy remains focused on enemy behavior

Game Programming

Delegates

- A delegate is a type-safe **reference to a method**
- Allows methods to be stored in variables
- Enables callbacks and flexible behavior
- Delegates are the technical basis of events
- In game engines (Unity, Unreal, Godot, etc.), delegates are useful for gameplay communication and UI callbacks

Game Programming

Simple Delegate Example

- The delegate type defines the method signature
- The variable stores a compatible method
- Calling the delegate invokes the referenced method

- **What Happens Here?**
  - MessageAction defines a method signature
  - currentAction stores a method reference
  - SayHello matches the delegate signature
  - Calling currentAction() invokes SayHello
  - The exact method can be changed at runtime

```
using UnityEngine;

public class DelegateExample : MonoBehaviour
{
    public delegate void MessageAction();
    private MessageAction currentAction;

    private void Start()
    {
        currentAction = SayHello;
        currentAction();
    }

    private void SayHello()
    {
        Debug.Log("Hello from a delegate!");
    }
}
```

Game Programming

Delegates with Parameters

- Delegates can define input parameters
- This makes them useful for health, score, damage, and inventory updates
- The receiver decides what to do with the data

```
using UnityEngine;

public class DamageDelegateExample : MonoBehaviour
{
    public delegate void DamageAction(int damageAmount);
    private DamageAction onDamage;

    private void Start()
    {
        onDamage = PrintDamage;
        onDamage(25);
    }

    private void PrintDamage(int amount)
    {
        Debug.Log("Damage taken: " + amount);
    }
}
```

Game Programming

Delegates as Flexible Behaviour

- Different methods can share the same delegate signature
- The selected behavior can change during runtime
- This is useful for AI actions, combat reactions, and UI callbacks
- Delegates reduce hardcoded method calls
- They are most useful when the caller should not know the concrete receiver

Game Programming

Multicast Delegates

- A delegate can reference multiple methods
- The += operator adds a method to the invocation list
- The -= operator removes a method
- Calling the delegate invokes all registered methods in order
- This is useful when many systems react to the same event

- One call triggers multiple reactions
- This is the basic idea behind many event systems

```
using UnityEngine;

public class MulticastDelegateExample : MonoBehaviour
{
    public delegate void GameAction();
    private GameAction onGameStarted;

    private void Start()
    {
        onGameStarted += SpawnPlayer;
        onGameStarted += PlayMusic;
        onGameStarted += ShowUI;
        onGameStarted?.Invoke();
    }

    private void SpawnPlayer() => Debug.Log("Player spawned");
    private void PlayMusic() => Debug.Log("Music started");
    private void ShowUI() => Debug.Log("UI shown");
}
```

Game Programming

Safe Delegate Invocation

- A delegate variable may be null when nobody is subscribed
- Calling a null delegate causes a *NullReferenceException*
- The null-conditional operator (?) prevents this
- This syntax is common in modern C# event code

```
// Calls the delegate only if it is not null.
onGameStarted?.Invoke();

// Equivalent, but more verbose:
if (onGameStarted != null)
{
    onGameStarted.Invoke();
}
```

Game Programming

Delegates in Unity UI

- Unity buttons use callback-style event registration
- AddListener connects a method to a button click
- This is a practical example of event-driven programming

```
using UnityEngine;
using UnityEngine.UI;

public class ButtonExample : MonoBehaviour
{
    [SerializeField]
    private Button startButton;

    private void Start()
    {
        startButton.onClick.AddListener(StartGame);
    }

    private void StartGame()
    {
        Debug.Log("Game started");
    }
}
```

Game Programming

Built-In Delegate Types

- **Action**
  - Represents a method that returns void
  - Can have zero or more parameters
  - Avoids declaring custom delegate types
  - Multiple reactions can be registered

```
using System;
using UnityEngine;

public class ActionExample : MonoBehaviour
{
    private Action onCollected;

    private void Start()
    {
        onCollected += PlayCollectSound;
        onCollected += AddScore;
        onCollected?.Invoke();
    }

    private void PlayCollectSound() => Debug.Log("Sound played");
    private void AddScore() => Debug.Log("Score added");
}
```

Game Programming

Built-In Delegate Types

- **Action<T>**
  - Represents a void method with one parameter
  - Action<int> is useful for score, health, ammo, or experience changes
  - Additional generic parameters allow more values

```
using System;
using UnityEngine;

public class ActionWithParameterExample : MonoBehaviour
{
    private Action<int> onScoreChanged;

    private void Start()
    {
        onScoreChanged += UpdateScoreUI;
        onScoreChanged?.Invoke(100);
    }

    private void UpdateScoreUI(int score)
    {
        Debug.Log("Score: " + score);
    }
}
```

Game Programming

Built-In Delegate Types

- **Func**
  - Func is a delegate that returns a value
  - The final generic type is the return type
  - Func is useful for calculations, queries, and strategy-like behavior
  - It is less common for events because events usually notify rather than compute

  - Func Example:
    - The delegate receives two integers and returns one integer

```
using System;
using UnityEngine;

public class FuncExample : MonoBehaviour
{
    private Func<int, int, int> calculateDamage;

    private void Start()
    {
        calculateDamage = AddWeaponBonus;
        int damage = calculateDamage(10, 5);
        Debug.Log("Damage: " + damage);
    }

    private int AddWeaponBonus(int baseDamage, int bonus)
    {
        return baseDamage + bonus;
    }
}
```

Game Programming

Built-In Delegate Types

- **Predicate**
  - Predicate<T> returns a bool
  - It is commonly used for filtering collections
  - Useful for inventory, enemy selection, target validation, and quests
  - It is equivalent in purpose to Func<T, bool>

```
using System;
using System.Collections.Generic;
using UnityEngine;

public class PredicateExample : MonoBehaviour
{
    private void Start()
    {
        List<int> damages = new List<int> { 5, 10, 20, 30 };
        Predicate<int> isHighDamage = damage => damage >= 20;
        List<int> highDamages = damages.FindAll(isHighDamage);

        foreach (int damage in highDamages)
            Debug.Log(damage);
    }
}
```

Game Programming

Events

- An event is a restricted form of a delegate
- Other classes can **subscribe and unsubscribe**
- Other classes cannot directly invoke the event
- The declaring class keeps control of when the event occurs
- Events are ideal for **gameplay communication**

```
// Delegate:
public Action OnPlayerDied;

// Event:
public event Action OnPlayerDied;
```

Game Programming

Events Example

- **PlayerHealth Event**
  - PlayerHealth owns the health state
  - It publishes changes without knowing the UI, audio, or game-over systems
  - The event carries the new health value

```
using System;
using UnityEngine;

public class PlayerHealth : MonoBehaviour
{
    public event Action<int> OnHealthChanged;
    public event Action OnPlayerDied;

    [SerializeField]
    private int health = 100;

    public void TakeDamage(int amount)
    {
        health -= amount;
        OnHealthChanged?.Invoke(health);

        if (health <= 0)
            OnPlayerDied?.Invoke();
    }
}
```

Game Programming

Events Example

- **Reacting to Health Changes**
  - The UI listens to the health event
  - It does not need to be called directly by the player
  - Subscribing and unsubscribing follow Unity lifecycle methods

  - **Why Unsubscribe in OnDisable?**
    - Prevents callbacks to inactive or destroyed objects
    - Avoids duplicated subscriptions when components are re-enabled
    - Reduces event-related memory leaks
    - Prevents confusing bugs where a method runs multiple times
    - This is one of the most important event rules

```
using UnityEngine;

public class HealthUI : MonoBehaviour
{
    [SerializeField]
    private PlayerHealth playerHealth;

    private void OnEnable()
    {
        playerHealth.OnHealthChanged += UpdateHealthText;
    }

    private void OnDisable()
    {
        playerHealth.OnHealthChanged -= UpdateHealthText;
    }

    private void UpdateHealthText(int health)
    {
        Debug.Log("Health UI updated: " + health);
    }
}
```

Game Programming

Events

- **Common Event Bug**
  - The same method can be registered multiple times
  - This often happens when a component is enabled repeatedly
  - Always pair subscription with unsubscription

```
private void OnEnable()
{
    playerHealth.OnHealthChanged += UpdateHealthText;
}

// Missing OnDisable means this method may be registered again later.
// The symptom: one health change updates the UI multiple times.
```

Game Programming

Events Example

- **Game-Over Controller**
  - The game-over controller reacts to player death
  - PlayerHealth does not directly know about the UI panel or time scale
  - The reaction is separated from the health model

```
using UnityEngine;

public class GameOverController : MonoBehaviour
{
    [SerializeField]
    private PlayerHealth playerHealth;
    [SerializeField]
    private GameObject gameOverPanel;

    private void OnEnable() => playerHealth.OnPlayerDied += ShowGameOver;
    private void OnDisable() => playerHealth.OnPlayerDied -= ShowGameOver;

    private void ShowGameOver()
    {
        gameOverPanel.SetActive(true);
        Time.timeScale = 0f;
    }
}
```

Game Programming

Events Example

- **Audio Feedback without Modifying PlayerHealth**
  - The audio system listens to the death event
  - No changes are required inside PlayerHealth
  - This illustrates the open/closed principle in practice

```
using UnityEngine;

public class PlayerAudioFeedback : MonoBehaviour
{
    [SerializeField]
    private PlayerHealth playerHealth;
    [SerializeField]
    private AudioSource deathSound;

    private void OnEnable() => playerHealth.OnPlayerDied += PlayDeathSound;
    private void OnDisable() => playerHealth.OnPlayerDied -= PlayDeathSound;

    private void PlayDeathSound()
    {
        deathSound.Play();
    }
}
```

Game Programming

Events

- **Benefits**
  - PlayerHealth contains health logic only
  - UI, audio, and game-over behavior are separate modules
  - New reactions can be added without changing the publisher
  - Testing and debugging are easier because responsibilities are narrower
  - Scene composition becomes more flexible

Game Programming

Event-Driven Enemy System

- **Enemy Death Event**
  - The enemy announces its own death
  - The event passes the enemy instance to the listeners
  - This allows listeners to inspect the object that died

- **Why Pass *this*?**
  - The listener knows *which* enemy died
  - The score system can inspect enemy data
  - The spawn system can remove the enemy from a list
  - The achievement system can count enemy kills
  - The event becomes more informative

```
using System;
using UnityEngine;

public class EnemyHealth : MonoBehaviour
{
    public event Action<EnemyHealth> OnEnemyDied;
    [SerializeField]
    private int health = 50;

    public void TakeDamage(int amount)
    {
        health -= amount;

        if (health <= 0)
        {
            OnEnemyDied?.Invoke(this);
            Destroy(gameObject);
        }
    }
}
```

Game Programming

Event-Driven Enemy System

- **Score System** Listening to Enemy Death
  - This works for one known enemy
  - It is useful for explaining the mechanism
  - It becomes less scalable with many dynamically spawned enemies

  - **Limitation of Direct Enemy References**
    - The score listener must know every enemy it should observe
    - Dynamically spawned enemies require additional registration logic
    - Scene setup can become fragile
    - A centralized **event channel** can reduce reference complexity
    - This leads to **ScriptableObject** event channels

```
using UnityEngine;

public class ScoreOnEnemyDeath : MonoBehaviour
{
    [SerializeField]
    private EnemyHealth enemy;
    [SerializeField]
    private int scoreValue = 100;
    private int score;

    private void OnEnable() => enemy.OnEnemyDied += HandleEnemyDied;
    private void OnDisable() => enemy.OnEnemyDied -= HandleEnemyDied;

    private void HandleEnemyDied(EnemyHealth deadEnemy)
    {
        score += scoreValue;
        Debug.Log("Score: " + score);
    }
}
```

Game Programming

ScriptableObject Event Channels

- Decoupling senders and receivers through assets

- **Event Channel**
  - An event channel is an object whose job is to **broadcast a specific event**
  - In Unity, ScriptableObjects are useful as shared event assets
  - Senders reference the channel, not the receivers
  - Receivers reference the same channel, not the senders
  - This is especially useful with prefabs and **dynamically spawned objects**

Game Programming

ScriptableObject Event Channels

- **Enemy Defeated Event Channel**
  - The channel exposes an event and a method for raising it
  - The asset can be assigned in the Inspector

```
using System;
using UnityEngine;

[CreateAssetMenu(menuName = "Events/Enemy Defeated Event")]
public class EnemyDefeatedEventChannel : ScriptableObject
{
    public event Action<int> OnEnemyDefeated;

    public void RaiseEvent(int scoreValue)
    {
        OnEnemyDefeated?.Invoke(scoreValue);
    }
}
```

Game Programming

ScriptableObject Event Channels

- **Enemy Raising the Event**
  - The enemy does not know the score manager
  - It only knows the shared event channel asset
  - This works well for enemy prefabs

```
using UnityEngine;

public class Enemy : MonoBehaviour
{
    [SerializeField]
    private int health = 50;
    [SerializeField]
    private int scoreValue = 100;
    [SerializeField]
    private EnemyDefeatedEventChannel defeatedEvent;

    public void TakeDamage(int amount)
    {
        health -= amount;
        if (health <= 0)
        {
            defeatedEvent.RaiseEvent(scoreValue);
            Destroy(gameObject);
        }
    }
}
```

Game Programming

ScriptableObject Event Channels

- **ScoreManager Listening to the Event Channel**
  - The score system subscribes to the shared event channel
  - Any enemy using the same channel can affect the score

- **Advantages of Event Channels**
  - No direct enemy-to-score-manager reference
  - Works well with prefabs
  - Multiple systems can listen
  - Designers can assign channels in the Inspector
  - Useful for UI, audio, analytics, and game state

```
using UnityEngine;

public class ScoreManager : MonoBehaviour
{
    [SerializeField]
    private EnemyDefeatedEventChannel enemyDefeatedEvent;
    private int score;

    private void OnEnable() => enemyDefeatedEvent.OnEnemyDefeated += AddScore;
    private void OnDisable() => enemyDefeatedEvent.OnEnemyDefeated -= AddScore;

    private void AddScore(int value)
    {
        score += value;
        Debug.Log("Score: " + score);
    }
}
```

Game Programming

ScriptableObject Event Channels

- **Advantages of Event Channels**
  - No direct enemy-to-score-manager reference
  - Works well with prefabs
  - Multiple systems can listen
  - Designers can assign channels in the Inspector
  - Useful for UI, audio, analytics, and game state

- **Possible Drawbacks**
  - Event flow can become harder to trace
  - Too many event channels can create complexity
  - Debugging can become harder
  - Events should represent meaningful domain actions

```
using UnityEngine;

public class ScoreManager : MonoBehaviour
{
    [SerializeField]
    private EnemyDefeatedEventChannel enemyDefeatedEvent;
    private int score;

    private void OnEnable() => enemyDefeatedEvent.OnEnemyDefeated += AddScore;
    private void OnDisable() => enemyDefeatedEvent.OnEnemyDefeated -= AddScore;

    private void AddScore(int value)
    {
        score += value;
        Debug.Log("Score: " + score);
    }
}
```

Game Programming

Lambdas

- A lambda is an **anonymous function**
- Can be stored in a delegate
- Useful for **short, local behavior**
- Common in UI callbacks and collection queries
- **Improves compactness** but can reduce readability if overused
- => reads as “goes to”

```
() => Debug.Log("Hello");

int x => x * 2;

(a, b) => a + b;
```

Game Programming

Lambda Example

- **Unity Button**
  - Useful when the callback is short
  - The lambda is registered as a listener

```
using UnityEngine;
using UnityEngine.UI;

public class LambdaButtonExample : MonoBehaviour
{
    [SerializeField]
    private Button playButton;

    private void Start()
    {
        playButton.onClick.AddListener(() =>
        {
            Debug.Log("Play button clicked");
        });
    }
}
```

Game Programming

Lambdas

- **Passing Parameters with Lambdas**
  - Button.onClick expects a method with no parameters
  - The lambda wraps a method call that has parameters (LoadLevel)
  - This avoids creating many tiny wrapper methods

```
using UnityEngine;
using UnityEngine.UI;

public class LevelButton : MonoBehaviour
{
    [SerializeField]
    private Button level1Button;
    [SerializeField]
    private Button level2Button;

    private void Start()
    {
        level1Button.onClick.AddListener(() => LoadLevel(1));
        level2Button.onClick.AddListener(() => LoadLevel(2));
    }

    private void LoadLevel(int levelIndex)
    {
        Debug.Log("Loading level " + levelIndex);
    }
}
```

Game Programming

Lambdas

- **Lambda in Collection Filtering**
  - LINQ queries frequently use lambdas
  - LINQ = Language Integrated Query
  - This is useful for selecting nearby enemies, available items, or valid targets

```
using System.Collections.Generic;
using System.Linq;
using UnityEngine;

public class EnemyQueryExample : MonoBehaviour
{
    private void Start()
    {
        List<int> enemyDistances = new List<int> { 3, 7, 12, 20 };
        var nearbyEnemies = enemyDistances
            .Where(distance => distance <= 10)
            .ToList();

        foreach (int distance in nearbyEnemies)
            Debug.Log("Nearby enemy: " + distance);
    }
}
```

Game Programming

Lambdas

- **Lambda in Inventory Filtering**
  - Filters are compact and expressive
  - This approach is useful for UI inventories and item search

```
using System.Collections.Generic;
using System.Linq;
using UnityEngine;

public class InventoryExample : MonoBehaviour
{
    private void Start()
    {
        List<string> items = new() { "Potion", "Sword", "Health Potion", "Shield" };
        var potions = items.Where(item => item.Contains("Potion")).ToList();

        foreach (string item in potions)
            Debug.Log(item);
    }
}
```

Game Programming

Event Queues

- Some actions should happen later, not immediately
- Events may need predictable ordering
- Audio, UI, analytics, tutorials, and achievements often consume queued events
- Queues help avoid chaotic direct communication
- They are useful in larger systems

Game Programming

Simple Event Queue

- **Creating the Event Queue**
  - The queue stores actions
  - The Update method processes all queued actions in order

```
using System;
using System.Collections.Generic;
using UnityEngine;

public class GameEventQueue : MonoBehaviour
{
    private readonly Queue<Action> eventQueue = new Queue<Action>();

    public void Enqueue(Action gameEvent)
    {
        eventQueue.Enqueue(gameEvent);
    }

    private void Update()
    {
        while (eventQueue.Count > 0)
            eventQueue.Dequeue().Invoke();
    }
}
```

Game Programming

Simple Event Queue

- **Using the Event Queue**
  - Each event is added as a small action
  - The queue controls when the events are processed

```
using UnityEngine;

public class EventQueueUser : MonoBehaviour
{
    [SerializeField]
    private GameEventQueue eventQueue;

    private void Start()
    {
        eventQueue.Enqueue(() => Debug.Log("Enemy defeated"));
        eventQueue.Enqueue(() => Debug.Log("Score updated"));
        eventQueue.Enqueue(() => Debug.Log("Achievement checked"));
    }
}
```

Game Programming

Component-Based Design

- Unity is component-based
- GameObjects are composed from components
- Each component should have a clear responsibility
- Avoid large monolithic scripts
- Composition is often better than inheritance
- Components should communicate through direct references, interfaces, or events depending on the situation

Game Programming

Component-Based Design

- **Bad Example: One Giant Player Script**
  - Movement
  - Jumping
  - Shooting
  - Health
  - Animation
  - Audio
  - UI
  - Input handling
  - Inventory
  - Interaction

Game Programming

Component-Based Design

→ **Better: Multiple Components**

- PlayerInputReader reads input
- PlayerMovement applies movement
- PlayerHealth stores health state
- PlayerWeapon handles firing
- PlayerAnimator updates animations
- PlayerAudio plays feedback
- PlayerInteraction handles usable objects
- PlayerInventory stores collected items

Game Programming

Component-Based Design

- **Small Movement Component**
  - Movement logic is isolated
  - Input does not need to be hardcoded inside this component

```
using UnityEngine;

public class PlayerMovement : MonoBehaviour
{
    [SerializeField]
    private float speed = 5f;

    public void Move(Vector2 input)
    {
        Vector3 movement = new Vector3(input.x, 0f, input.y);
        transform.position += movement * speed * Time.deltaTime;
    }
}
```

Game Programming

Component-Based Design

- **Separate Input Component**
  - The input component reads keyboard input
  - It delegates movement to PlayerMovement
  - Later, this can be replaced by gamepad, touch, or AI input

```
using UnityEngine;

public class PlayerInputReader : MonoBehaviour
{
    [SerializeField]
    private PlayerMovement movement;

    private void Update()
    {
        Vector2 input = new Vector2(
            Input.GetAxisRaw("Horizontal"),
            Input.GetAxisRaw("Vertical")
        );
        movement.Move(input);
    }
}
```

Game Programming

Component-Based Design

- **Responsibility Separation**
  - PlayerInputReader reads input (from keyboard, mouse, joystick, etc.)
  - PlayerMovement moves the player
  - The movement system can be driven by player input, AI, or network state
  - Smaller components are easier to test and replace
  - The GameObject becomes an assembly of focused behaviors

Game Programming

Data-Driven Design

- **Behavior is controlled by data** rather than hardcoded values
- Designers can tweak values without changing code
- **Balancing becomes easier**
- Unity supports this well through ScriptableObjects
- Data-driven design improves iteration speed

Game Programming

Data-Driven Design Example

- **WeaponData**
  - WeaponData is an asset, not a scene object
  - Different weapons can share one Weapon script and use different data

```
using UnityEngine;

[CreateAssetMenu(menuName = "Game/Weapon Data")]
public class WeaponData : ScriptableObject
{
    public string weaponName;
    public int damage;
    public float fireRate;
    public AudioClip fireSound;
    public GameObject projectilePrefab;
}
```

Game Programming

Data-Driven Design Example

- **Weapon using WeaponData**
  - The weapon reads from the assigned data asset
  - Creating a new weapon variant may only require a new asset

```
using UnityEngine;

public class Weapon : MonoBehaviour
{
    [SerializeField]
    private WeaponData weaponData;

    public void Fire()
    {
        Debug.Log("Firing " + weaponData.weaponName);
        Debug.Log("Damage: " + weaponData.damage);

        Instantiate(weaponData.projectilePrefab, transform.position, transform.rotation);
    }
}
```

Game Programming

Data-Driven Design

- **Benefits of ScriptableObject Data**
  - Reusable configuration assets
  - Cleaner code with fewer hardcoded values
  - Faster balancing and iteration
  - Multiple weapon variants can share one script
  - Designer-friendly workflows through the Inspector

Game Programming

Architecture Pitfalls

- **When Events Become Too Much**
  - Too many events make flow hard to understand
  - Poor naming causes confusion
  - Hidden dependencies can appear
  - Debugging event chains can be difficult
  - Use events for meaningful domain changes, not every method call

Game Programming

Summary

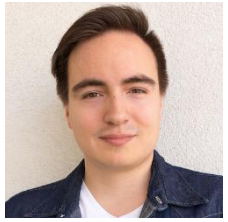
- Delegates allow methods to be passed as values
- Events enable safe decoupled communication
- Lambdas provide compact local behavior
- ScriptableObjects support data-driven design
- Good architecture makes games easier to extend, debug, and optimize
- Game patterns provide good architectural foundation => see next slide set

Thanks for your Attention

FH Oberösterreich
University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design
Digital Arts
Interactive Media



Dr. Andreas Riegler

Professor of Interactive Systems

Digital Media Department

E Andreas.Riegler@fh-hagenberg.at